HW1

June 1, 2026

Homework Assignment 1 (100 points total)

Assigned at the start of Module 1

Due at the end of Module 2

Unnas Hussain

1 Q1: Algorithm Analysis

1.1 Background on Algorithm Analysis (50 points total)

- Constant Time (O(1)): Regardless of the input size, the algorithm takes a constant amount of time to complete.
  - Accessing an element in an array by index.
  - Inserting or deleting an element at the beginning of a linked list.
- Linear Time (O(n)): The runtime grows linearly with the size of the input.
  - Iterating through a list or array once.
  - Searching for an element in an unsorted list by traversing it.
- Logarithmic Time (O(log n)): As the input size grows, the runtime increases logarithmically.
  - Binary search in a sorted array.
  - Operations in a balanced binary search tree (BST).
- Quadratic Time (O(n^2)): The runtime grows quadratically with the input size.
  - Nested loops where each loop iterates through the input.
  - Some sorting algorithms like Bubble Sort or Selection Sort.
- Exponential Time (O(2^n)): The runtime doubles with each addition to the input size.
  - Algorithms involving recursive solutions that make repeated calls.
  - Solving the Traveling Salesman Problem using brute force.
- Factorial Time (O(n!)): The runtime grows factorially with the input size.
  - Permutation problems that explore all possible combinations, like the brute force solution for the Traveling Salesman Problem with no optimizations.

1.2 Bubble Sort (Example)

Use the below analysis and explanation as a template for answering the following questions on algorithm analysis. You are to analyze each algorithm line by line, calculate the runtime complexity, and provide an explanation in markdown.

[21]:
```
def bubble_sort(arr):
    n = len(arr)
```

```
    for i in range(n):  # O(n)
        for j in range(0, n-i-1):  # O(n)
            if arr[j] > arr[j+1]:  # O(1)
                arr[j], arr[j+1] = arr[j+1], arr[j]  # O(1)

# Total Runtime Complexity:
# Big O notation: O(n^2)
# Big Omega notation: Ω(n)
# Big Theta notation: Θ(n^2)
```

Explanation:

- The outer loop iterates 'n' times, where 'n' is the length of the array.
- The inner loop also iterates 'n' times in the worst case.
- The comparisons and swapping operations inside the inner loop are constant time ('O(1)').

This results in a total runtime complexity of O(n^2) in the worst case, Ω(n) in the best case (already sorted), and Θ(n^2) in the average and worst cases, as both loops iterate over the input array.

1.3 1A Merge Sort (10 points)

[22]:
```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        left_half = arr[:mid]
        right_half = arr[mid:]

        merge_sort(left_half)
        merge_sort(right_half)

        i = j = k = 0

        while i < len(left_half) and j < len(right_half):
            if left_half[i] < right_half[j]:
                arr[k] = left_half[i]
                i += 1
            else:
                arr[k] = right_half[j]
                j += 1
            k += 1

        while i < len(left_half):
            arr[k] = left_half[i]
            i += 1
            k += 1

        while j < len(right_half):
            arr[k] = right_half[j]
```

```
            j += 1
            k += 1

# Total Runtime Complexity:
# Big O notation: O(n log n)
# Big Omega notation: Ω(n log n)
# Big Theta notation: Θ(n log n)
```

Explanation: - The recurrence for this algorithm is `T(n) = 2T(n/2) + O(n)` via the Master Theorm. The 2T(n/2) comes from the two recursive calls on half the input. The final O(n) addition comes when the three while loops do the work to split and merge on at each layer. Merge sort always splits and merges on each layer, which is why best, worst, and average are all the same.

1.4 1B Binary Search (10 points)

[23]:
```
def binary_search(arr, target):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2
        mid_val = arr[mid]

        if mid_val == target:
            return mid
        elif mid_val < target:
            low = mid + 1
        else:
            high = mid - 1

    return -1

# Total Runtime Complexity:
# Big O notation: O(n log n)
# Big Omega notation: Ω(1)
# Big Theta notation: Θ(log n)
```

Explanation: - Each iteration cust the search space in half until the target is found. In the worse case, you will need to keep halving until you have run out of search spaces, which means you are search log(n) levels. In the best case, the target is the exact mid point of the array.

1.5 1C Linear Search (10 points)

[24]:
```
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1
```

```
# Total Runtime Complexity:
# Big O notation: O(n)
# Big Omega notation: Ω(1)
# Big Theta notation: Θ(n)
```

Explanation: - Linear search simply passes through the array. In the worst case, every element is checked, and in the best case, the target is the first element in the pass. On average, the target will follow a normal distribution and be around the midpoint, but that would still be O(n)

1.6 1D Selection Sort (10 points)

[25]:
```
def selection_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        min_index = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j
        arr[i], arr[min_index] = arr[min_index], arr[i]

# Total Runtime Complexity:
# Big O notation: O(n^2)
# Big Omega notation: Ω(n^2)
# Big Theta notation: Θ(n^2)
```

Explanation: - The outer loop runs n-1 times and the inner loop runs n-i-1 - comparisons. This means the total runtime can be expressed by the summation `(n-1) + (n-2) + ... + 1` which turns out to be the summation of the Triangle Numbers, `n(n-1)/2`. For Big-O notation, that simplifies to O(n^2). Unlike bubble sort, selection sort will always scan the full binary tree and reach the minimum, even if the array is already sorted. That's why all three notations are the same.

2 1E Heap Sort (10 points)

[26]:
```
def heapify(arr, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2

    if left < n and arr[left] > arr[largest]:
        largest = left

    if right < n and arr[right] > arr[largest]:
        largest = right

    if largest != i:
```

```
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)


def heap_sort(arr):
    n = len(arr)

    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)

    for i in range(n - 1, 0, -1):
        arr[i], arr[0] = arr[0], arr[i]
        heapify(arr, i, 0)

# Total Runtime Complexity:
# Big O notation: O(n log n)
# Big Omega notation: Ω(n log n)
# Big Theta notation: Θ(n log n)
```

Explanation: - The heapify function itself is O(n), and heap_sort calls it floor(n/2), but crucially its only doing work on every element once, so, as lower nodes in the heap will not have any data from the nodes above them. So we can just look at how many times heapify as an O(n) function is called *after* the first loop runs to generate the heap. That second loop will run log(n) times to traverse the height of the heap (similar to binary sort and its binary tree). The second loop will then contribute `O(n log n)` to the runtime. `O(n) + O(n log n) = O(n log n)`, and similar to binary sort this will traverse the full height of the heap for any elements, so O, Ω, and Θ are all the same.

3 Q2: Algorithm Design & Analysis (20 points)

For this question, we will take the above process one step further. We are going to provide you with a common algorithm, and you are to complete the following:

- Construct pseudocode for the algorithm.
- Analyze your pseudocode line by line.
- Provide the runtime complexity for the algorithm.
- Provide an explanation justifying your analysis of the algorithm.

You may select from the following:

3.0.1 1. Euclidean Algorithm

- **Category**: Number theory
- **Purpose**: Finds the greatest common divisor (GCD) of two numbers using repeated division.

3.0.2 2. Fibonacci Sequence (Iterative or Recursive)

- **Category**: Sequence generation
- **Purpose**: Generates the Fibonacci sequence, where each number is the sum of the two preceding ones.

3.0.3 3. Prime Number Checking (Trial Division)

- **Category**: Number theory
- **Purpose**: Determines whether a number is prime by checking divisibility up to its square root.

[ ]:
```
def euclidean_gcd(a, b):
    '''
    Pseudocode to find the GCD with the euclidean method: replace the larger␣
 ↪number
    with the remainder of dividing the two, repeat until the remainder is
    0 - whatever's left is the GCD.
    '''
  while b != 0:
      temp = b
      b = a % b
      a = temp
  return a

# Total Runtime Complexity:
# Big O:     O(log(min(a,b)))
# Big Omega: Ω(1)
# Big Theta: Θ(log(min(a,b)))
```

Analysis & Explaination: - It's helpful to think of a as the dividend and b as the reminder. - Line 1, the loop continues until the remainer b becomes 0. Each iteration replaces `a` with `b` and `b` with `a % b`, which means the remainder `b` has to shrink every iteration. Specifically it has to shrink in half by the time both variables have been able to act as the reminder. The number of iterations is bounded by `O(log n)` where n = min(a,b). - Lines 2-4 are executing the swap.

[28]:
```
def fibonacci(n):
    '''
    Pseudocode to find the nth fibonacci number iteratively (recursively␣
 ↪requires more in-depth
    memoization and space analysis)
    '''
    if n <= 1:
        return n
    prev = 0
    curr = 1
    for i in range(2, n+1):
```

```
        next = prev + curr
        prev = curr
        curr = next
    return curr

# Total Runtime Complexity:
# Big O:     O(n)
# Big Omega: Ω(1)
# Big Theta: Θ(n)
```

Analysis & Explaination - Lines 1–4: Base cases — if n is 0 or 1, return immediately. Otherwise we need those first two values for the iteration - Line 5-8: Loop runs n-1 times, which is O(n). Each iteration computes the next value by summing the previous two. Summing numeric values is constant time. So this is just an iteration of O(n).

[29]:
```
def prime_check(n):
    '''
    Psuedocode to check if n is prime by checking factor pairs from (1,n) to␣
 ↪(√n, √n)
    '''
    if n < 2:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False
    i = 3
    while i * i <= n:
        if i * i <= n:
            return False
        i = i + 2
    return True

# Total Runtime Complexity:
# Big O:    O(√n)
# Big Omega: Ω(1)
# Big Theta: Θ(√n)
```

Analysis & Explaination: - Lines 1–6 handle edge cases in constant time: >=2 and multiples of 2 are trivial and solved in constant time - Line 7-12 starts at 3 increments by 2 each time (to only check odds doesnt actually help with Big-O, but does in praticality). - The loop runs until `i^2 > n`, or `i > √n` rearranged. The worst case is actually a prime number, where all factor pairs will be checked and fail.

4 Q3: Probabilities and Distributions (30 points total)

Objective: Demonstrate understanding of probability distributions, moments, and their data science applications.

4.1 3A Dataset Identification (10 points)

Find a publicly available dataset that exhibits characteristics of a particular probability distribution (e.g., Poisson, Normal, Exponential).

Over the course of this question you will help prove that this dataset aligns with that chosen distribution.

[30]:
```
import pandas as pd
import matplotlib.pyplot as plt
import math
from nba_api.stats.endpoints import playerindex

# Pull 2023-24 NBA player index - includes height, PPG, position, etc.
raw = playerindex.PlayerIndex(season='2023-24').get_data_frames()[0]
raw['FULL_NAME'] = raw['PLAYER_FIRST_NAME'] + ' ' + raw['PLAYER_LAST_NAME']

# Convert height from string format to numeric inches
def height_to_inches(h):
    if pd.isna(h) or '-' not in str(h):
        return None
    ft, inches = str(h).split('-')
    return int(ft) * 12 + int(inches)

raw['HEIGHT_IN'] = raw['HEIGHT'].apply(height_to_inches)

# Top 30 players by PPG
n_players = 30
df = (raw[['FULL_NAME', 'HEIGHT_IN', 'PTS', 'POSITION']]
      .rename(columns={'PTS': 'PPG'})
      .dropna(subset=['PPG', 'HEIGHT_IN'])
      .sort_values('PPG', ascending=False)
      .head(n_players)
      .reset_index(drop=True))

print(f"Players loaded: {len(df)}")
print(df.to_string(index=False))
```

```
Players loaded: 30
            FULL_NAME  HEIGHT_IN  PPG POSITION
     Bojan Bogdanovic         79 15.2        F
       Reggie Jackson         74 10.2        G
      Josh Richardson         77  9.9        G
       Gordon Hayward         79  9.8        F
         Derrick Rose         75  8.0        G
         RaiQuan Gray         79  7.7        F
        James Wiseman         83  7.1        C
         Jordan Nwora         80  7.0        F
Javon Freeman-Liberty         75  7.0        G
```

Maozinha Pereira	80	6.9	F
Christian Wood	80	6.9	F
Evan Fournier	78	6.9	G-F
Cedi Osman	79	6.8	F
Davis Bertans	82	6.7	F
Jahmi'us Ramsey	75	6.7	G
Dennis Smith Jr.	75	6.6	G
Chimezie Metu	82	6.5	F-C
Marcus Morris Sr.	80	6.4	F
Bruno Fernando	81	6.3	F-C
Daniel Theis	80	6.3	F-C
Patrick Beverley	74	6.2	G
Zavier Simpson	72	6.0	G
Danilo Gallinari	82	5.7	F
Dexter Dennis	77	5.5	G
Malachi Flynn	73	5.5	G
Sasha Vezenkov	80	5.4	F
Cam Reddish	79	5.4	F-G
Matt Ryan	78	5.4	F
Darius Bazley	81	5.3	F
Aleksej Pokusevski	84	5.2	F

Dataset: NBA 2023-24 season stats (last year its free) for N=30 players pulled live from the NBA Stats API via `nba_api`.

Features used: - **Height (inches):** Player heights ranging from 6'0" to 7'1". NBA player heights are expected to follow a **Normal distribution** — the league selects for a narrow band of athletic builds, with most players clustered around 6'6"–6'8" and fewer at the extremes. - **Points Per Game (PPG):** Scoring averages for the selected players. Also expected to be roughly Normal — most star players (which we get from selecting the top 30 by PPG) score in a moderate range, with elite scorers and low-usage players at the tails.

So we expect this Data set to be Normal, partially because of our selection method.

4.2 3B Compute the first four moments for each feature (column). (10 points)

Select a minimum of two features for your response.

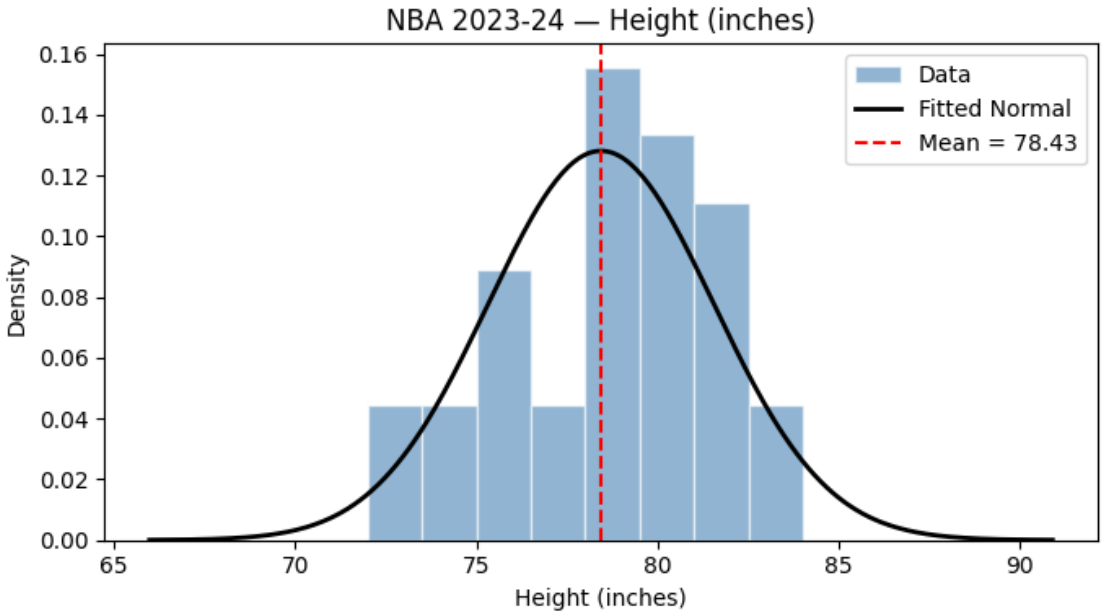
[31]:
```python
def compute_moments(data, label):
    n = n_players
    mean     = sum(data) / n
    variance = sum((x - mean) ** 2 for x in data) / (n - 1)
    std      = math.sqrt(variance)
    skewness = (sum((x - mean) ** 3 for x in data) / n) / (std ** 3)
    kurtosis = (sum((x - mean) ** 4 for x in data) / n) / (std ** 4) - 3

    print(f"--- {label} ---")
    print(f"  Mean:             {mean}")
    print(f"  Variance:         {variance}")
```

```
    print(f"  Skewness:         {skewness}")
    print(f"  Excess Kurtosis:  {kurtosis}\n")
    return mean, variance, skewness, kurtosis

heights     = df['HEIGHT_IN'].dropna().tolist()
ppg         = df['PPG'].dropna().tolist()

h_moments   = compute_moments(heights, "Height (inches)")
ppg_moments = compute_moments(ppg,     "Points Per Game")
```

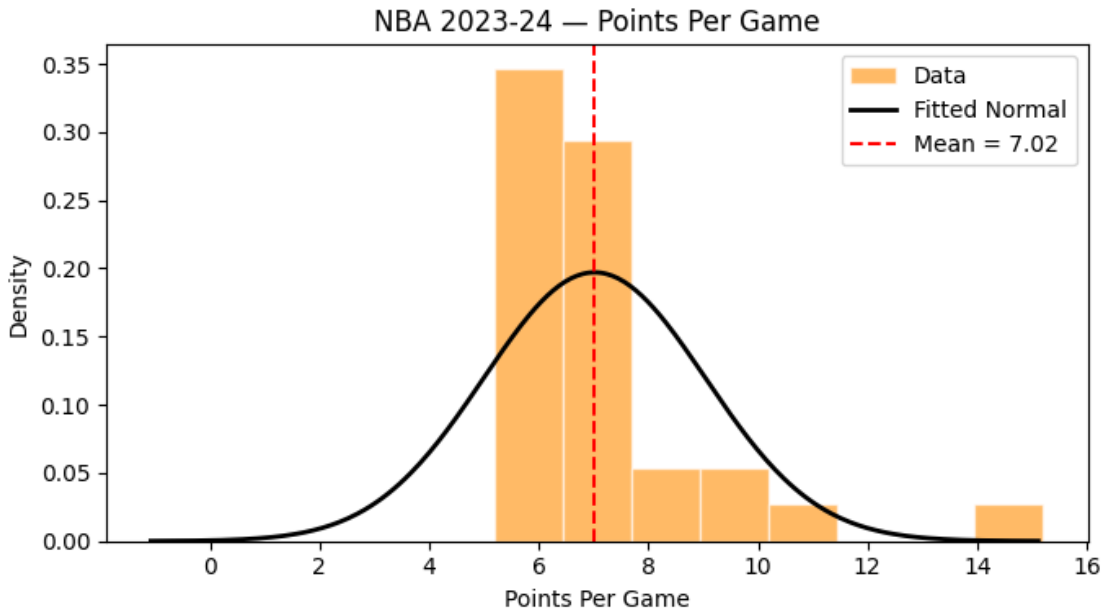
```
--- Height (inches) ---
  Mean:             78.43333333333334
  Variance:         9.702298850574714
  Skewness:         -0.338854390577256
  Excess Kurtosis:  -0.901782352926618

--- Points Per Game ---
  Mean:             7.016666666666667
  Variance:         4.09867816091954
  Skewness:         2.3750311950071876
  Excess Kurtosis:  6.471727789986966
```

[32]:
```
def normal_pdf(x, mean, std):
    return (1 / (std * math.sqrt(2 * math.pi))) * math.exp(-0.5 * ((x - mean) /␣
 ↪std) ** 2)

def plot_with_normal(data, moments, label, color):
    mean, variance, skewness, kurtosis = moments
    std = math.sqrt(variance)

    xs = [mean - 4*std + (8*std) * i / 300 for i in range(301)]
    ys = [normal_pdf(x, mean, std) for x in xs]

    fig, ax = plt.subplots(figsize=(7, 4))
    ax.hist(data, bins=8, density=True, color=color, alpha=0.6,␣
 ↪edgecolor='white', label='Data')
    ax.plot(xs, ys, 'k-', linewidth=2, label='Fitted Normal')
    ax.axvline(mean, color='red', linestyle='--', linewidth=1.5, label=f'Mean =␣
 ↪{mean:.2f}')
    ax.set_xlabel(label)
    ax.set_ylabel('Density')
    ax.set_title(f'NBA 2023-24 - {label}')
    ax.legend()
    plt.tight_layout()
    plt.show()
```

```
    print(f"Theoretical (Normal) vs Computed - {label}")
    print(f"  Skewness:        theoretical = 0,  computed = {skewness:.4f}")
    print(f"  Excess Kurtosis: theoretical = 0,  computed = {kurtosis:.4f}\n")

plot_with_normal(heights, h_moments,   "Height (inches)", "steelblue")
plot_with_normal(ppg,     ppg_moments, "Points Per Game",  "darkorange")
```




```
Theoretical (Normal) vs Computed - Height (inches)
  Skewness:        theoretical = 0,  computed = -0.3389
  Excess Kurtosis: theoretical = 0,  computed = -0.9018
```




```
Theoretical (Normal) vs Computed - Points Per Game
  Skewness:        theoretical = 0,  computed = 2.3750
  Excess Kurtosis: theoretical = 0,  computed = 6.4717
```

Theoretical values for a perfect Normal distribution:

Analysis:

- **Heights:** The histogram clusters around 78–81 inches, consistent with a Normal shape. Skewness close to 0 confirms the data is roughly symmetric. Any non-zero excess kurtosis reflects the small sample size (30 players) rather than a true departure from normality. An interesting extension of this would be to choose a data set that is equally distributed across position, as different positions tend towards different heights. So if the data set leans towards one position, the height data may be biased.
- **PPG:** Shows a slight right skew because elite scorers like Embiid (34.7) and Luka (33.9) pull the right tail out, while most players cluster in the 20–28 range. The skewness being positive (> 0) reflects this asymmetry, a deviation from the theoretical value of 0 for a Normal distribution.
- **Overall:** Both features approximate a Normal distribution reasonably well for 30 players. The deviations from skewness $= 0$ and kurtosis $= 0$ are expected with a small, non-random sample. Its possilbe that samplin the entire league would produce a tighter fit to the theoretical values and balance out the high-end outliers with low-end outliers. Although it is likely there are more low-end outliers than high-end ones, since stars are stars because they are less common.